

Docker Commands & Practical Notes

Compiled from the uploaded pages

S.No.	Docker Command / Topic	Meaning / Use
1	Docker info	Shows Docker system information.
2	docker images	View list of all Docker images.
3	docker run <image-name>	Create and run a container from an image.
4	docker ps -a	Check list of all containers.
5	docker run -it <image-name>	Run container in interactive (-i) + terminal (-t) mode.
6	ls	Check all directories.
7	mkdir <directory-name>	Make a new directory.
8	docker stop <container-name> docker stop <container-id>	Stop a running container.
9	docker start <container-name> docker start <container-id>	Start an existing stopped container.
10	docker pull <image-name>:<image-version> Example: docker pull mysql:8.0	Download/pull an image from Docker registry.
11	docker run -d <image-name>	Run image in detached mode (background).
12	docker run --name <container-name> -d <image-name>	Give the container a custom name and run in detached mode.
13	docker rmi <image-name> docker rmi <image-id>	Remove an image.
14	docker rm <container-name> docker rm <container-id> kisi Docker image k sabhi container ek sath remove kar ho to . docker rm -f \$(docker ps -a -q --filter "ancestor=ubuntu")	Remove a container.
15	docker run -d --name <container-custom-name> -e <container-current-name> Example: docker run -d --name tarunsql -e MYSQL_ROOT_PASSWORD=1234 mysql custome name docker run -d --name tarunsql mysql:latest -e MYSQL_ROOT_PASSWORD=1234 mysql	Define an environment variable. Common MySQL example: MYSQL_ROOT_PASSWORD.

Docker command

	docker logs tarunsql docker rm <container Id> Ek se zyada environment variables bhi de sakte ho: <pre> </> Bash docker run -d --name tarunsql ^ -e MYSQL_ROOT_PASSWORD=1234 ^ -e MYSQL_DATABASE=college ^ -e MYSQL_USER=tarun ^ -e MYSQL_PASSWORD=abc123 ^ mysql </pre>	
16	Port Binding docker run -p 8080:3306 <image-name> Format: hostPort:containerPort	Map a host port to a container port.
17	docker run -d --name mysql-latest -p 8080:3306 -e MYSQL_ROOT_PASSWORD=secret mysql:8.0 docker run -d --name shivamsql -p 8080:3306 -e MYSQL_ROOT_PASSWORD=12345 mysq	Example combining port binding, name, detached mode and environment variable.
18	docker logs <container-id>	Troubleshooting: view container logs.
19	docker exec -it <container-id>/bin/bash docker exec -it <container-id>/bin/sh	Open an interactive shell inside a running container.
20	docker network ls	Fetch/list all Docker networks.
21	docker network create <network-name>	Create a Docker network.
22	docker run -d ^ -p 27017:27017 ^ -e MONGO_INITDB_ROOT_USERNAME=tarun ^ -e MONGO_INITDB_ROOT_PASSWORD=tarun2805 ^ --name tarunMongo ^ --network mongo-network ^ mongo	Run a MongoDB container with port binding, root credentials, custom name and a user-defined network.
23	Docker Compose	Used to set up multiple/container-based services and their configuration.

Docker command

24	<code>docker-compose -f <filename.yml> up -d</code>	Start services in detached mode.
25	<code>docker-compose -f <filename.yml> down</code>	Stop/remove the Compose containers.
26	<code>docker network ls</code> <code>docker ps</code> <code>docker logs <container-id></code> Docker docs / Read docker file <code>docker hub search <image></code>	Useful Docker checking, troubleshooting and documentation commands.
27	<code>docker build -t tarunapp: 1.4</code> <code>docker run ^</code> <code>-p 3000:3000 -e ^</code> <code>MONGO_URI="mongodb://tarun:tarun2805@host.docker.internal:27017/?authSource=admin" tarunapp:1.4</code>	Build an image from a Dockerfile and run the created image.
28	Push the image on Docker Hub <code>docker build -t tarun2805/nodeapp .</code> <code>docker login</code> <code>docker push tarun2805/nodeapp</code> <code>docker logout</code> <code>docker pull tarun2805/nodeapp:latest</code>	Build, log in, push an image to Docker Hub, log out, and pull the published image.
29	Docker volume <code>docker run -it -v "C:\Users\Tarun2\Desktop\datafolder:/test/data" ubuntu</code> <code>docker exec -it 62c0f892fccd /bin/bash</code>	Persistent storage for Docker containers.
30	<code>docker run -v volume_Name:<container_dir_path></code>	Named volume: mount a named Docker volume into a container.
31	<code>docker run -v <mount_path></code>	Anonymous volume: Docker creates/manages the volume automatically.
32	<code>docker run -v <host_dir_path>:<mount_dir_path></code>	Bind mount: map a host directory to a container directory.
33	Docker network drivers: 1. Bridge — default network driver 2. Host 3. Null	Common Docker network driver types.

🔥 Sabse important 6 yaad rakho:

```
0    → Success
1    → General Error
126  → Command execute nahi hui
127  → Command nahi mili
137  → SIGKILL / often OOM
139  → Segmentation Fault
143  → SIGTERM
```



Aur exact error jaanne ke liye `docker logs <container>` sabse useful command hai.

```
Exited (0) → normally successfully finished
Exited (1) → error/failure
Exited (137) → process was killed (often SIGKILL/OOM)
Exited (255) → abnormal/error exit
```



dikhte hain.

1. Pehle STATUS ko samjho

```
docker ps -a
```



Example:

```
STATUS
Up 10 seconds
Exited (0) 2 minutes ago
Exited (1) 5 minutes ago
Created
Restarting (1) 10 seconds ago
Paused
Dead
```

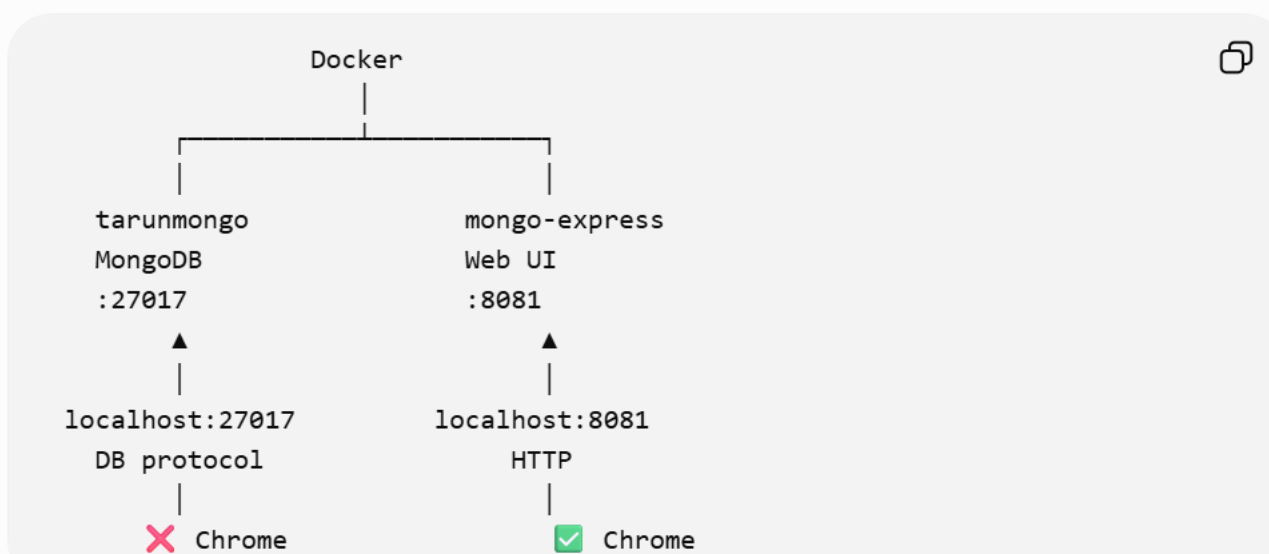


Ye sab container states/statuses hain.

2. Docker Container ki main states



State	Meaning	Example
Created	Container create ho gaya, but start nahi hua	Created
Running / Up	Container ka main process currently running hai	Up 2 minutes
Exited	Main process terminate ho gaya	Exited (0)
Paused	Container ke processes temporarily freeze/suspend hain	Paused
Restarting	Docker container ko automatically restart kar raha hai	Restarting (1)
Dead	Container terminate hua, lekin Docker proper cleanup nahi kar paya	Dead
Removing	Container remove ho  process mein hai	Removing



Docker command

Command	Kya karega?	Deep Explanation	Share
<code>docker info</code>	Docker system ki information dikhata hai	Docker Engine, containers, images, storage driver, CPUs, memory etc. ki information deta hai. Docker properly kaam kar raha hai ya nahi check karne ke liye useful hai.	
<code>docker version</code>	Docker Client/Server version dikhata hai	Client aur Docker Engine ki versions check karta hai. Version mismatch/debugging me useful.	
<code>docker --help</code>	Docker commands ki help	Docker ke available commands/options dekhne ke liye.	
<code>docker <command> --help</code>	Particular command ki help	Example: <code>docker run --help</code> se <code>docker run</code> ke saare options milenge.	
<code>docker images</code>	Local images ki list	Machine me downloaded/built Docker images dikhata hai.	
<code>docker image ls</code>	Same as <code>docker images</code>	Modern Docker syntax.	
<code>docker images -a</code>	All images/layers	Intermediate/dangling images bhi dekh sakte ho.	

2. Docker Image Commands

Document me `docker pull`, `docker images` aur `docker rmi` diye gaye hain.

`fd394eda-e437-4134-ac73-12d9851...`

Command	Kya karega?	Deep Explanation	
<code>docker pull mysql:8.0</code>	Docker Hub se MySQL image download	Registry se image local machine par download karta hai.	
<code>docker pull ubuntu</code>	Ubuntu image download	Default tag generally <code>latest</code> hota hai if available.	
<code>docker images</code>	Images list	Local machine me available images.	
<code>docker image inspect mysql</code>	Image ki detailed information	Layers, architecture, config, entrypoint etc.	
<code>docker history mysql</code>	Image layers/history	Image kin layers se bani hai ye dikhata hai.	
<code>docker rmi mysql</code>	Image delete	Image ko local machine se remove karta hai.	

Docker command

<code>docker rmi <image-id></code>	ID se image delete	Image name ke instead ID use kar sakte ho.
<code>docker rmi -f mysql</code>	Force remove image	Agar normal remove nahi ho raha ho to force remove.
<code>docker image prune</code>	Unused images clean	Unused/dangling images remove karta hai.

Image aur Container ka relation

Docker Hub



`docker pull`



IMAGE



`docker run`



CONTAINER



Docker command

Command	Kya karega?	Important Point
<code>docker run ubuntu</code>	Ubuntu container create/run	Container ka main process terminate hote hi container exit ho sakta hai.
<code>docker run -it ubuntu</code>	Interactive terminal	<code>-i</code> = stdin open, <code>-t</code> = terminal allocate.
<code>docker run -d nginx</code>	Background me container	<code>-d</code> = detached mode.
<code>docker run --name mymongo mongo</code>	Custom name	Container ko automatically generated name ke bajay <code>mymongo</code> naam milega.
<code>docker run -d --name mymongo mongo</code>	Named + background	Production/dev environment me useful.
<code>docker run --rm ubuntu</code>	Exit ke baad container automatically delete	Temporary containers ke liye useful.



Docker command

Option	Full Meaning	Kya karta hai?	
<code>-i</code>	Interactive	STDIN open rakhta hai	
<code>-t</code>	TTY	Terminal interface provide karta hai	
<code>-it</code>	Interactive + TTY	Container ke andar terminal open karne ke liye	
<code>-d</code>	Detached	Container background me run hota hai	
<code>--rm</code>	Remove automatically	Container stop hone ke baad remove	
<code>--name</code>	Custom name	Container ka naam set	
<code>-e</code>	Environment variable	Container ke andar environment variable set	
<code>-p</code>	Publish port	Host port ko container port se map	
<code>-v</code>	Volume/mount	Storage mount	



5. Container List Commands

Document me `docker ps -a` diya gaya hai.

 fd394eda-e437-4134-ac73-12d9851...

Command	Kya karega?
<code>docker ps</code>	Sirf currently running containers
<code>docker ps -a</code>	Running + stopped + exited containers
<code>docker ps -q</code>	Sirf container IDs
<code>docker ps -aq</code>	All containers ke IDs
<code>docker ps --filter "status=running"</code>	Sirf running containers
<code>docker ps --filter "status=exited"</code>	Sirf exited containers
<code>docker ps --filter "name=mongo"</code>	Mongo naam wale containers

6. Container Lifecycle Commands

Document me `stop` , `start` , `rm` diye gaye hain.

 fd394eda-e437-4134-ac73-12d9851...

Command	Kya karega?	Deep Explanation	
<code>docker stop mycontainer</code>	Container gracefully stop	Main process ko stop signal deta hai.	
<code>docker start mycontainer</code>	Stopped container start	Existing container ko dobara start karta hai.	
<code>docker restart mycontainer</code>	Stop + start	Container restart karta hai.	
<code>docker kill mycontainer</code>	Immediately kill	Forcefully process terminate karta hai.	
<code>docker pause mycontainer</code>	Processes freeze	Container ke processes temporarily suspend.	
<code>docker unpause mycontainer</code>	Processes resume	Paused container ko continue karta hai.	



Docker command

```
docker rm mycontainer
```

Container delete

Container remove karta hai.

```
docker rm -f  
mycontainer
```

Force delete

Running container ko bhi force remove kar sakta hai.

```
docker rename old new
```

Container rename

Existing container ka naam change.

stop vs kill

```
docker stop
```

↓

Graceful shutdown

↓

Application ko cleanup ka chance



```
docker kill
```

↓

Immediate termination



Isliye normal situation me:

```
-p HOST_PORT:CONTAINER_PORT
```

Command	Meaning
-p 8080:3306	Host 8080 → Container 3306
-p 27017:27017	Host 27017 → Container 27017
-p 3000:3000	Host 3000 → Container 3000
-p 8081:8081	Host 8081 → Container 8081

10. docker logs

Document me troubleshooting ke liye `docker logs <container-id>` diya hai.

 fd394eda-e437-4134-ac73-12d9851...

Command	Kya karega?
<code>docker logs mysql</code>	Container ke logs
<code>docker logs -f mysql</code>	Live logs follow
<code>docker logs --tail 50 mysql</code>	Last 50 lines
<code>docker logs -t mysql</code>	Timestamp ke saath logs
<code>docker logs --since 10m mysql</code>	Last 10 minutes ke logs

11. docker exec

Document me container ke andar shell open karne ke liye `docker exec` diya gaya hai.

 fd394eda-e437-4134-ac73-12d9851...

Correct syntax

 **Bash**

`docker exec -it <container> /bin/bash`



ya

 **Bash**

`docker exec -it <container> /bin/sh`



Command	Kya karega?
<code>docker exec -it mongo /bin/bash</code>	Running container me Bash
<code>docker exec -it mongo /bin/sh</code>	Running container me Shell



Docker command

<code>docker exec -it mongo /bin/sh</code>	Running container me Shell
<code>docker exec mongo ls</code>	Container ke andar <code>ls</code> execute
<code>docker exec mongo env</code>	Environment variables
<code>docker exec mongo pwd</code>	Current directory
<code>docker exec -it mysql mysql -u root -p</code>	Container ke MySQL client ko run

Important

`docker exec` sirf running container par use hota hai.

```
docker exec
  ↓
Running container
```

Stopped container par normally `exec` nahi chalega.

12. docker inspect

Ye document ke basic commands ke saath bahut important related command hai.

Command	Kya dekhega?
<code>docker inspect container</code>	Container ki complete configuration
<code>docker inspect image</code>	Image details
<code>docker inspect network</code>	Network details
<code>docker inspect volume</code>	Volume details

Example:

</> Bash

```
docker inspect tarunsql
```

Docker command

```
// bash
```

```
docker inspect tarunsql
```

Isse tumhe mil sakta hai:

- IP address
- environment variables
- mounts
- ports
- networks
- image
- container configuration

13. Container Resource Monitoring

Command	Kya karega?
<code>docker stats</code>	Running containers ka CPU/RAM/network usage
<code>docker stats mysql</code>	Specific container ka resource usage
<code>docker top mysql</code>	Container ke processes
<code>docker port mysql</code>	Port mappings

Example:

```
</> Bash
```

```
docker stats
```

Output roughly:

CONTAINER	CPU %	MEM USAGE
mysql	2%	300MB
nodeapp	1%	120MB
mongo	4%	500MB

14. Docker Network Commands

Document me `docker network ls` aur `docker network create` diye hain.

 fd394eda-e437-4134-ac73-12d9851...

Command	Kya karega?	
<code>docker network ls</code>	Docker networks list	
<code>docker network create mynetwork</code>	New network create	
<code>docker network inspect mynetwork</code>	Network ki details	
<code>docker network connect mynetwork container</code>	Existing container ko network me connect	
<code>docker network disconnect mynetwork container</code>	Network se disconnect	
<code>docker network rm mynetwork</code>	Network delete	
<code>docker network prune</code>	Unused networks delete	

Docker command

</> **Bash**



```
docker network create mongo-network
```

Then:

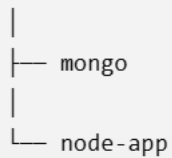
</> **Bash**



```
docker run -d \  
--name mongo \  
--network mongo-network \  
mongo
```

Ab same Docker network me doosra container Mongo ko uske **container name** se access kar sakta hai.

mongo-network



Node app se:

Docker command

```
docker run -d ^  
-p 27017:27017 ^  
-e MONGO_INITDB_ROOT_USERNAME=tarun ^  
-e MONGO_INITDB_ROOT_PASSWORD=tarun2805 ^  
--name tarunMongo ^  
--network mongo-network ^  
mongo
```

Isko table se samjho:

Part	Meaning
<code>docker run</code>	Container create + start
<code>-d</code>	Background
<code>-p 27017:27017</code>	MongoDB port expose
<code>-e MONGO_INITDB_ROOT_USERNAME=tarun</code>	Root username
<code>-e MONGO_INITDB_ROOT_PASSWORD=tarun2805</code>	Root password

Docker command

<code>-e MONGO_INITDB_ROOT_USERNAME=tarun</code>	Root username
<code>-e MONGO_INITDB_ROOT_PASSWORD=tarun2805</code>	Root password
<code>--name tarunMongo</code>	Custom container name
<code>--network mongo-network</code>	User-defined network
<code>mongo</code>	MongoDB image

Connection:

```
Host
localhost:27017
  ↓
Docker
  ↓
tarunMongo
  ↓
MongoDB :27017
```

16. Docker Compose

Document me Docker Compose ko multiple container-based services aur configuration ke liye diya gaya hai.  fd394eda-e437-4134-ac73-12d9851...

Command	Kya karega?	
<code>docker compose up</code>	Services create/start	
<code>docker compose up -d</code>	Background me start	
<code>docker compose down</code>	Services/containers stop/remove	
<code>docker compose ps</code>	Compose services ki status	
<code>docker compose logs</code>	Services logs	
<code>docker compose logs -f</code>	Live logs	
<code>docker compose restart</code>	Services restart	
<code>docker compose stop</code>	Services stop	
<code>docker compose restart</code>	Services restart	
<code>docker compose stop</code>	Services stop	
<code>docker compose start</code>	Services start	
<code>docker compose build</code>	Images build	
<code>docker compose pull</code>	Required images pull	

Document me old-style:

</> Bash

```
docker-compose -f <filename.yml> up -d
docker-compose -f <filename.yml> down
```

17. Dockerfile se Image Build

Document me:

</> Bash

```
docker build -t tarunapp:1.4
```

ka concept diya hai.  fd394eda-e437-4134-ac73-12d9851...

Normally correct command:

</> Bash

```
docker build -t tarunapp:1.4 .
```

Part	Meaning
docker build	Dockerfile se image build
-t	Tag/name assign
tarunapp	Image name

Docker command

:1.4

Image tag/version

.

Current directory ko build context

Example:

</> Bash

```
docker build -t mynodeapp:1.0 .
```

Then:

</> Bash

```
docker images
```

me:

```
mynodeapp    1.0
```



dikhega.

18. Build → Run

Document me image build karke run karne ka example diya gaya hai.

 fd394eda-e437-4134-ac73-12d9851...

</> Bash

```
docker build -t tarunapp:1.4 .
```

Then:

</> Bash

```
docker run -p 3000:3000 tarunapp:1.4
```

Docker command

Then:

</> Bash

```
docker run -p 3000:3000 tarunapp:1.4
```

Flow:

```
Dockerfile
  ↓
docker build
  ↓
Docker Image
tarunapp:1.4
  ↓
docker run
  ↓
Container
```

19. Docker Hub Commands

Document me login, push, logout aur pull diye gaye hain.

 fd394eda-e437-4134-ac73-12d985

Command	Kya karega?
<code>docker login</code>	Docker registry me login
<code>docker logout</code>	Logout
<code>docker push username/image</code>	Image registry par upload
<code>docker pull username/image</code>	Registry se image download
<code>docker tag image username/image:tag</code>	Image ko registry-compatible tag
<code>docker search nginx</code>	Docker Hub par image search

Docker command

Complete workflow

</> Bash

```
docker build -t tarun2805/nodeapp:1.0 .
```

</> Bash

```
docker login
```

</> Bash

```
docker push tarun2805/nodeapp:1.0
```

Another machine:

</> Bash

```
docker pull tarun2805/nodeapp:1.0
```

Then:

20. Docker Volume Commands

Document me Docker volumes, named volume, anonymous volume aur bind mount cover kiye gaye hain.  fd394eda-e437-4134-ac73-12d9851...

Volume ka purpose

Container normally **ephemeral** hota hai.

Agar container delete:

Container



Deleted



Container ke andar ka data bhi lose ho sakta hai



Volume:

Container



Volume



Persistent Data



21. Named Volume

Command	Meaning
<code>docker volume ls</code>	Volumes list
<code>docker volume create myvolume</code>	Named volume create
<code>docker volume inspect myvolume</code>	Volume details
<code>docker volume rm myvolume</code>	Volume delete
<code>docker volume prune</code>	Unused volumes remove

Example:

`</>` **Bash**

```
docker volume create mysql-data
```



Docker command

```
</> Bash  
  
docker volume create mysql-data
```

Then:

```
</> Bash  
  
docker run -d \  
--name mysql \  
-v mysql-data:/var/lib/mysql \  
-e MYSQL_ROOT_PASSWORD=1234 \  
mysql
```

Meaning:

```
mysql-data  
  ↓  
/var/lib/mysql  
  ↓  
MySQL container
```



Container delete hone ke baad bhi volume separately exist kar sakta hai.

22. Bind Mount

Document me:

```
</> Bash  
  
docker run -v <host_dir_path>:<mount_dir_path>
```

diya gaya hai.  fd394eda-e437-4134-ac73-12d9851...

Example:

```
</> Bash  
  
docker run -it -v "C:\Users\Tarun2\Desktop\datafolder:/test/data" ubuntu
```

Make sure...

Docker command

Meaning:

```
Windows
C:\Users\Tarun2\Desktop\datafolder
    ↓
    ↓ bind mount
    ↓
Container
/test/data
```

Host me file create:

```
hello.txt
```

Container me:

```
/test/data/hello.txt
```

23. Named Volume vs Anonymous Volume vs Bind Mount

Type	Syntax	Storage control	Typical Use
Named Volume	<code>-v mydata:/data</code>	Docker manages	Database data
Anonymous Volume	<code>-v /data</code>	Docker manages	Temporary/persistent unnamed storage
Bind Mount	<code>-v C:\folder:/data</code>	User controls host path	Development/code sharing

Easy memory trick

mydata:/data

↑

Named Volume

/data

↑

Anonymous Volume

C:\folder:/data

↑

Bind Mount